Tournament Management Platform

Master Plan, Part 2 of 4: Complete Sequential Roadmap (empty folder to deployment)

How to read: each phase is a table of ordered steps. **Step** IDs (e.g. B7) are referenced in *Depends on*. **Done when** is the verification point and completion criterion. Phases A to G are the foundation, H to AB the product modules, AC testing, AD deployment. Detail stays at planning level on purpose; coding instructions come when we implement each step.

Windows 11 notes that apply everywhere: use PowerShell or Git Bash consistently; avoid OneDrive-synced folders; keep paths short; set Git line endings once (B3); restart the terminal after installing Node/Git; allow Node through Windows Firewall only if prompted.

Phase index

Phase	Name	Steps
A	Requirement finalization and preparation	8
B	Development environment and GitHub repository	20
C	MongoDB Atlas and database initialization	14
D	Backend initialization and foundation	15
E	Frontend initialization and foundation	13
F	Database design finalization	8
G	Authentication, authorization and administrative bootstrap	15
H	Public website shell and tournament discovery	8
I	Tournament creation, configuration, rules and lifecycle	10
J	Player registration, approvals, waiting lists and withdrawal	10
K	Captain applications, selection, auction eligibility, team formation, manual assignment	9
L	Auction configuration and session initialization	6
M	Live auction interface and engine	12
N	Knockout fixture generation	5
O	League and round-robin fixture generation	3
P	Groups, standings, qualification and knockout progression	8
Q	Manual fixtures, scheduling, rescheduling and cancellation	5
R	Match status and manual result entry	6
S	CSV scorecard upload, parsing, validation, preview and confirmation	7
T	Result correction, history, standings and qualification recalculation	5

Phase	Name	Steps
U	Announcements and persistent notifications	5
V	Socket.IO real-time updates and fallback	5
W	Dashboards	3
X	Tournament archives and historical match pages	3
Y	Tournament reports and CSV/PDF exports	4
Z	Privacy controls, account lifecycle, suspensions and administrative workflows	5
AA	Audit-log viewing and administrative oversight	3
AB	Accessibility, responsive design, performance and final polish	4
AC	Testing and quality assurance (introduced throughout)	18
AD	Production deployment	15

Phase A: Requirement finalization and preparation

Pure planning. Output: a signed-off decision log in docs/.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
A1	Re-read the Blueprint and this plan; list every requirement with an ID (R-001...) in docs/requirements.md	Single requirement register	None	Every Blueprint section 2 to 22 item mapped to an ID
A2	Answer the open issues from Part 1 section 1.2	Decision log (docs/decisions.md)	A1	All 13 issues have a decision; BLOCKING ones resolved
A3	Confirm stack: TypeScript, monorepo, server-side sessions, extra collections	Architecture Decision Records (ADR-001..)	A2	ADRs written for stack, auth, repo layout
A4	Define MVP slicing: M1 to M8 milestones (Part 4); mark nothing as dropped	Release plan	A2	Every requirement assigned to a milestone
A5	Define module boundaries and which module owns which collection	Module map in docs/architecture.md	A3	No collection owned by two modules
A6	Draft state-machine tables for tournament, registration, lot, fixture, result, account request	Transition tables	A2	Each status has allowed transitions and actors
A7	Decide hosting shortlist and domain approach (same-site cookies)	Hosting shortlist	A3	Cookie strategy chosen
A8	List accounts/services needed (Part 4 section 3) and sign up for GitHub and Atlas	Accounts ready	A7	Can log in to each

Phase B: Development environment and GitHub repository

Windows 11 + VS Code. Keep the repo outside OneDrive-synced folders (e.g. C:\dev\tournament-platform).

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
B1	Check/install Node.js LTS, npm, Git for Windows, VS Code; record versions	Verified toolchain	A8	node -v, npm -v, git --version all print; versions noted in README
B2	Install VS Code extensions: ESLint, Prettier, Tailwind CSS IntelliSense, MongoDB for VS Code (optional), GitLens (optional)	Editor ready	B1	Extensions active
B3	Configure Git identity, default branch main, autocrlf=input, and a SSH key or GitHub credential manager	Git configured	B1	git config --list shows values; can authenticate to GitHub
B4	Create project folder C:\dev\tournament-platform and open in VS Code	Empty workspace	B1	Folder exists, opened
B5	git init; create root .gitignore (node_modules, .env*, dist, coverage, logs, .DS_Store) and .gitattributes/.editorconfig	Repo with ignores	B4	.env cannot be staged (verify with git status)
B6	Create root README.md (purpose, prerequisites, run commands placeholder)	README v0	B5	File committed
B7	Make the first commit on main (docs + ignores only)	Baseline commit	B5, B6	git log shows one commit
B8	Create the GitHub repository (private recommended), no auto-generated files	Remote repo	B3	Empty repo visible on GitHub
B9	Connect remote (origin) and push main	Code on GitHub	B7, B8	GitHub shows README, .gitignore

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
B10	Set branch strategy: main (always deployable) + short-lived feature/* branches + PRs; Conventional Commit messages; protect main	Workflow rules in docs/workflow.md	B9	Branch protection enabled (or documented if plan lacks it)
B11	Decision recorded: one repo for client and server (as per Blueprint)	ADR	A3	ADR committed
B12	Create client/ and server/ and docs/ folders (with .gitkeep)	Folder skeleton	B11	Tree matches Part 1 section 3
B13	Initialize server app: npm init in server/, set type=module, add TypeScript, tsx, @types/node, tsconfig; do not install runtime deps yet	server/package.json	B12	npx tsc --noEmit runs
B14	Initialize client app: npm create vite (react-ts) inside client/	client/ scaffold	B12	npm run dev serves default page
B15	Install dependencies separately in server/ and client/ (never mix lockfiles)	Two lockfiles	B13, B14	Each folder has own node_modules and package-lock.json
B16	Create .env.example in server/ and client/ with placeholder values only; copy to .env locally	Safe env templates	B15	.env ignored, .env.example committed
B17	Add ESLint + Prettier configs; scripts lint, format, typecheck per app	Quality tooling	B15	npm run lint passes in both apps
B18	Add root package.json with convenience scripts (dev via concurrently, lint, test, build)	One-command dev start	B17	npm run dev at root starts both
B19	Run both apps and confirm they work (Vite page loads; server placeholder logs listening)	Working skeleton	B18	Browser shows Vite page; server console shows port
B20	Commit and push skeleton in logical commits; add GitHub Actions CI stub (lint + typecheck)	CI green	B19	First green workflow run

Phase C: MongoDB Atlas and database initialization

Never put credentials in Git. Use separate DB users per environment.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
C1	Create MongoDB Atlas account and organization	Atlas access	A8	Logged in
C2	Create Atlas project "tournament-platform" and a free/lowest-cost cluster in a nearby region	Dev cluster	C1	Cluster state healthy
C3	Create a dedicated DB user for dev with a long generated password; least privilege (readWrite on dev DB only)	Dev DB user	C2	User listed; not the Atlas owner account
C4	Configure Network Access: add your current IP only (avoid 0.0.0.0/0 except documented temporary need)	IP allow-list	C2	Connection works from laptop only
C5	Choose database names: tournament_dev, tournament_test (memory), tournament_prod (later, separate cluster/user)	Naming convention	C2	Names recorded in docs
C6	Copy the connection string (SRV) into server/.env as MONGODB_URI (URL-encode special chars in password)	Local secret	C3, C4, B16	git status shows .env untracked

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
C7	Install mongoose and zod; write env validation (fail fast on missing vars)	Config module	C6	Server refuses to start with bad env
C8	Connect Mongoose and log success; handle connection errors/retry	DB connection	C7	Server logs Connected to MongoDB
C9	Test: connection check script + /health endpoint reporting db state (after D)	Verified connectivity	C8	Health returns db: up
C10	Plan transactions: confirm replica-set support (Atlas shared tier is replica set); list transactional operations (Part 3)	Transaction map	F1	Documented
C11	Plan seed strategy: scripts/seed.ts (idempotent, dev only, refuses to run in production) + demo data	Seed plan	F1	Plan in docs
C12	Plan admin provisioning: scripts/create-admin.ts reads ADMIN_EMAIL and one-time password from environment, creates the only admin, aborts if one exists	Admin bootstrap design	G1	Design reviewed
C13	Plan dev/prod separation (separate clusters, users, secrets, DB names; NODE_ENV guards)	Env separation	C5	No shared credentials
C14	Plan backups and restores: mongodump schedule, off-provider storage, restore drill; versioned migrations (migrate-mongo or simple scripts with schemaVersion field)	Backup + migration plan	C13	Runbook outline in docs/runbooks

Phase D: Backend initialization and foundation

Goal: a secure, observable, tested Express shell with no business features yet.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
D1	Install runtime deps: express, mongoose, zod, helmet, cors, cookie-parser, express-rate-limit, pino, pino-http, socket.io, argon2 or bcryptjs	Dependencies	B15, C7	npm ls clean
D2	Install dev deps: vitest, supertest, mongodb-memory-server, tsx, types	Test deps	D1	Installed
D3	Create folder structure (config, middleware, modules, sockets, scripts, utils)	Backend skeleton	D1	Matches Part 1 section 3
D4	Create app.ts (Express factory) and server.ts (listen + graceful shutdown)	App entry	D3	Server starts and stops cleanly
D5	Add config/env.ts with Zod validation and typed env access	Typed config	D4	Missing var causes startup error
D6	Add middleware: helmet, CORS allow-list from env, JSON body limit, cookie parser, request id, pino-http logging	Middleware stack	D4	Headers/logs verified
D7	Define API response and error conventions: {data} / {error:{code,message,details}}; AppError classes; central error handler; 404 handler	Error contract	D6	Unit test for error mapping
D8	Add Zod validation middleware (body, query, params)	Validation layer	D7	Invalid input returns 400 with details
D9	Add rate-limit middleware presets (auth, bids, admin, general)	Rate limit presets	D6	Limit triggers in test
D10	Add /api/health and /api/ready endpoints	Health checks	D5, C8	Returns ok with db status

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
D11	Set up session model + auth/role middleware shells (authenticate, requireRole, requireOwner) returning 401/403	Auth middleware skeleton	D7	Protected test route behaves
D12	Configure Socket.IO on the same HTTP server; cookie-based socket auth; namespaces /auction and /live (no handlers yet)	Socket foundation	D4	Test client connects
D13	Set up Vitest, Supertest and in-memory replica set helper; write first tests (health, error handler)	Test harness	D2, D10	npm test passes locally and in CI
D14	Verify first endpoint through browser/Postman/curl	End-to-end proof	D10	GET /api/health returns 200
D15	Commit; add backend README section (scripts, env vars)	Documented	D14	Pushed, CI green

Phase E: Frontend initialization and foundation

Can run in parallel with Phase D after D6.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
E1	Configure Tailwind correctly for the installed major version; verify with a utility class	Tailwind working	B14	Styled test element renders
E2	Install react-router, TanStack Query, react-hook-form + zod resolver, socket.io-client	Dependencies	E1	Installed
E3	Create folder structure (app, components, pages, features, services, hooks, types)	Frontend skeleton	E2	Matches Part 1
E4	Configure Vite env files (VITE_API_URL, VITE_SOCKET_URL) and a dev proxy to the API so cookies are same-origin in development	Env-aware URLs	E2	API calls work with no CORS errors
E5	Build centralized API client (fetch wrapper: credentials, JSON, error normalization, 401 handling)	api client	E4, D7	Unit tests for error mapping
E6	Create router with route groups: public, auth, player, organizer, admin; route guards driven by /auth/me	Routing	E3	Guarded routes redirect
E7	Create auth context/store (session bootstrap on refresh, logout)	Auth state	E5, E6	Refresh keeps login (after G)
E8	Build layouts (public shell with header/footer, dashboard shell with sidebar) with responsive nav	Layouts	E6	Looks right at 360px and 1280px
E9	Build design-system primitives: Button, Input, Select, Modal, Table, Badge, Toast, Tabs, Pagination, Skeleton	UI kit	E1	Storybook-like demo page
E10	Build state components: Loading, Empty, Error, Unauthorized/Forbidden, NotFound	State UX	E9	Used across demo page
E11	Build form helpers with Zod schemas shared/duplicated with server types	Forms	E7, E9	Sample form validates
E12	Wire frontend to backend: page calls /api/health and displays status	Connectivity proof	D14, E5	Page shows API ok
E13	Add Vitest + React Testing Library setup; first component tests	Frontend tests	E9	npm test passes, CI runs it

Phase F: Database design finalization

Design only: no model code until the order in Part 3 section 9 is reached. See Part 3 for full detail.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
F1	Take the 15 Blueprint collections and evaluate each against requirements	Evaluation table	A1	Each kept/changed with justification
F2	Evaluate additional collections (sessions, ruleRevisions, invites, auctionSessions, importJobs, resultRevisions, counters)	Additions list	F1	Each marked essential/optional
F3	Decide embed vs reference for every relationship (growth, query pattern, history, concurrency)	Relationship map	F2	Documented in Part 3
F4	Define unique/compound/partial indexes for integrity rules	Index plan	F3	Every integrity rule has index or transaction
F5	List transaction-sensitive workflows and their atomic units	Transaction list	F4	Reviewed
F6	Draw ERD (Mermaid in docs/erd.md)	ERD	F3	Renders in GitHub
F7	Fix schema implementation order (Part 3 section 9)	Build order	F6	Agreed
F8	Review design against workflow walkthroughs (registration, bid, undo, correction, delete account)	Design sign-off	F7	No workflow lacks a data home

Phase G: Authentication, authorization and administrative bootstrap

Tests are written alongside each step.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
G1	Create User model (normalized unique email, passwordHash, role, status, visibility settings) and Session model with TTL index	Models + indexes	F8, D14	Unique email enforced by test
G2	Implement password hashing service (Argon2id or bcrypt) with policy (min length, breached-pattern guard)	Hash service	G1	Unit tests; no plaintext stored
G3	Implement register (player) and login with generic error messages, throttling per IP and per email	Auth endpoints	G2, D9	Brute-force test throttled
G4	Implement session issue/validate/rotate; httpOnly Secure SameSite cookie; CSRF defence (Origin check + custom header)	Secure sessions	G3	Cookie flags verified in test
G5	Implement logout and logout-all; revocation when status becomes suspended	Revocation	G4	Revoked session returns 401
G6	Implement /auth/me and frontend bootstrap so refresh keeps login	Persistence	G4, E7	Refresh test passes (Playwright)
G7	Implement role-based authorization middleware and tournament-ownership guard	RBAC	D11	Permission matrix tests
G8	Create admin bootstrap script (single admin, env-supplied one-time password, forces password change, refuses duplicates); block admin creation via public API	Safe admin	G1, G2	Cannot self-register as admin; second admin impossible

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
G9	Implement OrganizerApplication model, submit endpoint, admin approve/reject with reason, status change notification hook	Organizer flow	G7, G8	Unapproved organizer cannot create tournament
G10	Implement account suspension (admin) with session revocation	Suspension	G5, G7	Suspended user locked out
G11	Implement AccountRequest model and admin-assisted recovery: user request -> admin verifies -> single-use short-lived token (hash stored) -> user sets password -> sessions revoked	Recovery	G2, G8	Token reuse/expiry rejected; audited
G12	Implement account deletion request (request only; processing in Phase Z)	Deletion request	G11	Request visible to admin
G13	Implement AuditLog model + append-only service (no update/delete API) and log all G actions	Audit v1	G1	Every G action produces audit row
G14	Frontend: register, login, logout, organizer application, admin approval queue (basic)	Auth UI	G6, G9	Manual walkthrough passes
G15	Permission test suite: visitor, player, unapproved organizer, approved organizer, admin versus every protected route	Role matrix tests	G7	Matrix table all green in CI

Phase H: Public website shell and tournament discovery

Prereq: G. Completion: visitors can browse everything public without logging in.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
H1	Tournament model (minimal read fields) + seed demo tournaments	Browsable data	F8, G1	Seed creates varied tournaments
H2	Public API: list with pagination, search, filters (status, format, date), sort	GET /tournaments	H1	Filters tested; indexes used
H3	Public API: tournament detail with section endpoints (overview, rules, teams, fixtures, standings, announcements)	Detail API	H2	Returns only public fields
H4	Homepage: upcoming, ongoing, recently completed, featured/updated; search bar	Home page	H2, E8	Loads with skeleton/empty states
H5	Discovery page with filter UI and URL-synced query params	Discover page	H4	Refresh keeps filters
H6	Tournament detail page with tabs and deep links (/t/:id/fixtures)	Detail page	H3, E6	Deep link refresh works
H7	Invite-only tournaments shown publicly with a registration-restricted badge	Visibility rule	H5	Invite-only listed for visitors
H8	Component + API tests; responsive check	Tests	H7	CI green

Phase I: Tournament creation, configuration, rules and lifecycle

Prereq: G (approved organizer), H. Completion: organizer can create, configure and move a tournament through lifecycle.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
I1	Complete Tournament model: basics, format, registration settings, rules object, status, schemaVersion	Model	H1	Validation + indexes tested
I2	Zod schemas for create/update with format-specific rule presets and sensible defaults	Validated config	I1	Defaults applied
I3	Lifecycle state machine service (Draft -> RegistrationOpen -> RegistrationClosed -> TeamFormation -> AuctionInProgress -> FixturesPublished -> Ongoing -> Completed; Cancelled from allowed states)	Transitions	A6, I1	Invalid transitions rejected (table-driven tests)
I4	Create/edit endpoints with ownership check; organizer-only draft editing UI	Create/edit	I3, G7	Other organizers get 403
I5	TournamentRuleRevision model; every rule edit records old/new/actor/time/reason	Rule history	I1	Revision rows created
I6	Impact-preview service (dry-run) for rule changes affecting fixtures, standings, qualification; hooks stubbed until those phases	Impact preview	I5	Preview returns affected counts
I7	Auction-parameter freeze guard (issue 1)	Guard	I5	Mid-auction edit rejected
I8	Registration deadline automation: timer + lazy check on access; reopen endpoint with audit	Auto close/reopen	I3	Closes at deadline after restart
I9	Cancel and reschedule endpoints (reason required) + audit + notification hook	Cancel/reschedule	I3	Reason mandatory
I10	Organizer UI: wizard, rules editor with history view, lifecycle controls	Organizer UI	I4, E10	Walkthrough passes

Phase J: Player registration, approvals, waiting lists and withdrawal

Prereq: I. Completion: concurrent registration cannot exceed capacity or duplicate.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
J1	Registration model with partial unique index on (tournament, player) for active statuses; counters doc for capacity	Model + index	F8, I1	Duplicate insert fails at DB level
J2	Invite model + invite-only gate (code/link redemption)	Invites	A2, J1	Invalid code rejected
J3	Register service in a transaction: eligibility, deadline, duplicate, capacity -> approved / pending / waitlisted with atomic waiting position	Register service	J1, J2	Concurrency test: N parallel requests, exactly cap approved
J4	Auto vs manual approval; organizer approve/reject with reason	Approvals	J3	Both modes tested
J5	Waiting list: organizer promotes player manually when a spot opens; audit each change	Waiting list	J3	Promotion respects capacity
J6	Withdrawal rules (pre-auction / pre-start), organizer exceptions, historical retention	Withdrawal	J3	Status-based, no deletes
J7	Reopen registration interplay with waiting list	Reopen	I8, J5	Tested
J8	Notification hook calls (stub until U)	Hooks	J4	Events emitted
J9	Player UI (register, status, withdraw) and organizer UI (queue, waiting list)	UI	J4, E10	Walkthrough passes

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
J10	Integration + concurrency tests	Tests	J3	Green in CI

Phase K: Captain applications, selection, auction eligibility, team formation, manual assignment

Prereq: J. Completion: teams exist with valid squads via manual assignment.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
K1	Add captain-interest fields to registration; player toggle during registration	Captain interest	J1	Stored and visible to organizer
K2	Team model + team creation (count, names); counters for squadCount and purse	Teams	F8, I1	Unique team name per tournament
K3	Organizer selects captains (final decision) and links captain to team; audit	Captain selection	K1, K2	Only approved registrants selectable
K4	Auction-eligibility decision endpoint (separate from captain selection)	Eligibility	K1	Only eligible can be auctioned later
K5	TeamAssignment model with history (method, actor, previous)	Assignments	F8, K2	Unique active assignment per player per tournament
K6	Manual assignment service in a transaction enforcing squad limit, eligibility, uniqueness	Manual teams	K5	Over-capacity rejected
K7	Publish team assignment notifications; public teams/squads tab	Publish teams	K6	Visible publicly
K8	Organizer UI for captains, eligibility, drag/assign teams	UI	K6	Walkthrough passes
K9	Tests: duplicate assignment, squad limit, ineligible player	Tests	K6	Green

Phase L: Auction configuration and session initialization

Prereq: K. Completion: a valid, frozen auction session ready to start.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
L1	AuctionSession model: purses, squad max, base price, increment, countdown, order, unsold policy, status, round counter, version	Model	F8	Schema tests
L2	Configuration endpoints and validation (all values numeric, positive, consistent with squad and pool sizes)	Config API	L1, K4	Invalid configs rejected
L3	Auction order: fixed list / random seeded (store seed) / manual	Ordering	L2	Deterministic with seed
L4	Start endpoint: validates, freezes config, creates AuctionLot per eligible player, initializes team purses, sets tournament status AuctionInProgress	Initialization	L2, I3	Transactional; repeat start rejected
L5	Organizer config UI with dry-run summary	UI	L4	Walkthrough
L6	Tests for config validation and initialization	Tests	L4	Green

Phase M: Live auction interface and engine

Highest-risk phase. Build and test the engine with no UI first. Completion: concurrent bids are safe; countdown survives restart.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
M1	AuctionLot and AuctionBid models (append-only bids, compensating undo events, roundNumber, outcome)	Models	F8, L4	Bids cannot be updated/deleted via API
M2	Bid service: single atomic conditional update (status, version, currentPrice) inside transaction; validates captain, round rule, increment X+Y, purse, squad capacity	Atomic bidding	M1	Concurrency test: 50 parallel bids -> exactly one winner per price step
M3	Done endpoint: organizer closes round, server sets countdown endsAt	Done	M2	Only organizer
M4	Countdown engine: server timer, persisted endsAt, auto-sell on expiry, cancel on bid/undo, recovery at boot, lazy check on access	Countdown	M3	Restart test resumes
M5	Undo-last-bid: compensating event restoring price/leader, cancels countdown, requires reason	Undo	M2	History intact; audited
M6	Sell / unsold handling, purse and squad updates, TeamAssignment creation in same transaction	Sale finalization	M2, K5	No double sale; purse never negative
M7	Unsold round endpoint and finalize endpoint with full squad validation	Rounds + finalize	M6	Final squads valid
M8	Socket.IO broadcasting of lot state to /auction room; snapshot on connect; reconnect resync	Realtime	M3, D12	Reconnect test passes
M9	Public auction room UI (read-only), captain bid UI, organizer control panel; display server-offset countdown	Auction UI	M8, E10	Visitors cannot bid
M10	Rate limits on bids; rejection messages for stale/conflicting bids	Abuse control	M2	Tested
M11	Admin-only post-finalization correction path (audited)	Exceptional correction	M7	Only admin; audited
M12	Full auction test suite (concurrency, countdown, undo, purse, squad) + simulated auction script	Tests	M1..M11	All green in CI

Phase N: Knockout fixture generation

Prereq: K or M (teams exist). Completion: bracket generated and progression-ready.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
N1	Fixture model with stage, round, slots, dependency refs (winnerOf/loserOf), status	Model	F8, K2	Indexes tested
N2	Bracket generator: seeding modes (random with stored seed, manual), byes for non-power-of-two	Generator	N1	Property tests for sizes 2..64
N3	Dependency graph resolution: next-round slots populated when confirmed results arrive	Progression logic	N1	Unit tests

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
N4	Manual-override record for bracket edits	Overrides	N3	Audited
N5	Bracket UI (public + organizer)	UI	N3	Renders byes/empty slots

Phase O: League and round-robin fixture generation

Prereq: N1. Completion: correct round-robin for any team count.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
O1	Circle-method generator (single/double round robin, odd teams with bye)	Generator	N1	Every pair meets expected times
O2	Home/away balance and round numbering	Balance	O1	Tests for 3..20 teams
O3	League table view hooking into standings (P)	Table UI	O1	Renders

Phase P: Groups, standings, qualification and knockout progression

Prereq: N, O. Completion: confirmed outcomes update standings and dependent fixtures.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
P1	Group draw (random seed stored / manual) and group fixtures	Groups	O1	Balanced groups
P2	Standings engine: pure function from confirmed results + rules; points presets; configurable tie-breakers (head-to-head, differential, etc. as supported)	Engine	A5, N1	Table-driven tests with edge cases
P3	Qualification evaluator (top N per group, best-of-rest options) with ties surfaced to organizer	Qualification	P2	Tie scenarios tested
P4	Qualification override with previous/new/reason; audit and snapshot	Overrides	P3	Audited
P5	Populate knockout fixtures after qualification confirmed	Group -> KO	P3, N3	Correct bracket seeding
P6	standingsSnapshots written on every confirm/correct/override/rule change	Snapshots	P2	Snapshot per event
P7	Wire I6 impact preview to real standings/fixture calculations	Impact preview	P2	Preview matches real outcome
P8	UI: standings tables, group views, qualification badges	UI	P2	Walkthrough

Phase Q: Manual fixtures, scheduling, rescheduling and cancellation

Prereq: N. Completion: organizer can fully edit schedules with audited reasons.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
Q1	Manual fixture create/edit with validation (teams belong to tournament, no impossible pairs)	Manual fixtures	N1	Validation tests
Q2	Schedule editing (date, time, venue) with reason on changes; timezone handling	Scheduling	Q1	History preserved
Q3	Postpone/cancel fixture statuses and their effect on progression	Statuses	Q1	Progression blocked appropriately
Q4	Participant notifications for fixture changes (hooks)	Hooks	Q2	Events emitted
Q5	Organizer schedule UI (calendar/list)	UI	Q2	Walkthrough

Phase R: Match status and manual result entry

Prereq: P, Q. Completion: confirmed manual results feed standings; unconfirmed data stays private.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
R1	MatchResult + ResultRevision models (draft/previewed/confirmed, immutable revisions)	Models	F8, N1	Revisions immutable
R2	Match status transitions (scheduled/ongoing/completed/postponed/cancelled); public sees status only	Status API	R1	No scores in public payload before confirm
R3	Manual result entry with validation and preview	Manual entry	R1	Invalid winners rejected
R4	Confirm endpoint: transaction updates result, fixture, standings, bracket, snapshot, audit	Confirmation	R3, P2	All-or-nothing
R5	Organizer UI for status and result entry	UI	R4	Walkthrough
R6	Tests: confirmation atomicity, public data leakage	Tests	R4	Green

Phase S: CSV scorecard upload, parsing, validation, preview and confirmation

BLOCKED until a real sample CSV exists. Do not invent columns.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
S1	Obtain a real representative scorecard CSV (and ideally 2 to 3 variants)	Sample files in docs/samples/ (sanitized)	A, R	Files received and reviewed
S2	Document the column map, types, and meanings from the sample	Format spec	S1	Spec signed off by you
S3	Build importer adapter interface (parse -> normalize -> validate -> map) with the first adapter for the spec	Isolated parser	S2	Unit tests from sample
S4	Upload endpoint: type/size validation, in-memory parse, ImportJob with preview and row errors	Upload API	S3	Malformed files rejected safely

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
S5	Preview UI highlighting errors, unsupported fields and inline fixes	Preview UI	S4	Walkthrough
S6	Confirm step: promote ImportJob data to MatchResult; supported stats only; unreviewed imports never affect standings	Confirmation	S4, R4	Standings unaffected before confirm
S7	Fuzz and malformed tests (encoding/BOM, delimiters, huge, formula chars)	Tests	S4	Green

Phase T: Result correction, history, standings and qualification recalculation

Prereq: R, P. Completion: corrections preserve originals and update everything downstream.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
T1	Correction endpoint requiring reason; new immutable revision; original preserved	Correction	R4	Original retrievable
T2	Recalculate standings and qualification in the same transaction; compare snapshots	Recalc	T1, P6	Diff stored
T3	Downstream fixture/bracket impact detection with organizer confirmation if bracket would change	Downstream impact	T2, N3	No silent bracket change
T4	Correction UI with impact preview and history timeline	UI	T3	Walkthrough
T5	Tests for chains of corrections and bracket impacts	Tests	T3	Green

Phase U: Announcements and persistent notifications

Can start after J; finish after T.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
U1	Notification model with indexes (recipient, read, created) and TTL/retention policy	Model	F8	Index tested
U2	Central notify() service with event catalogue (Blueprint section 11 list) and recipient resolution	Service	U1	Each event type has template
U3	Replace hooks from I, J, K, M, Q, R, T with notify() calls	Wiring	U2	Each event produces notification
U4	Announcement model, publish/pin/history, notify participants	Announcements	U1	Pinned shows first
U5	Notification API (paginated, mark read, mark all read) + UI bell and page	API/UI	U1	Unread count correct

Phase V: Socket.IO real-time updates and fallback

Prereq: M8, U. Completion: updates arrive live and degrade gracefully.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
V1	Namespaces/rooms: auction:, tournament:, user:; auth on handshake; authorization per room	Room design	D12	Unauthorized join rejected
V2	Events: lot state, bids, countdown ticks (end time based), fixture/status/result published, notifications	Event catalogue	V1	Documented
V3	Reconnection: client resync via REST snapshot; version numbers to detect gaps	Resilience	V2	Reconnect test
V4	Polling fallback via TanStack Query with configurable interval (env/admin setting)	Fallback	V2	Works with sockets blocked
V5	Load test with many read-only spectators	Load check	V3	Acceptable on target host

Phase W: Dashboards

Prereq: respective feature phases. Completion: each role has a complete workspace.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
W1	Player dashboard: registrations, statuses, teams, fixtures, notifications, privacy settings	Player	J, K, U	Walkthrough
W2	Organizer dashboard: owned tournaments, queues, auction, fixtures/results, announcements, private activity log, exports	Organizer	I..U	Walkthrough
W3	Admin dashboard: organizer queue, users, all tournaments, requests, suspended items, audit logs, interventions	Admin	G, Z	Walkthrough

Phase X: Tournament archives and historical match pages

Prereq: R, T. Completion: completed tournaments permanent and browsable.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
X1	Finalize endpoint: validates all results, freezes tournament, writes final snapshot, publishes archive	Finalization	T3	Freeze enforced
X2	Public archive list and tournament history pages	Archive UI	X1	Completed visible
X3	Historical match detail page with revision history (public parts only)	Match page	R	Respects visibility

Phase Y: Tournament reports and CSV/PDF exports

Prereq: P, R. Completion: all export types download correctly and safely.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
Y1	Export service with permission checks and history-visibility filtering	Service	X1	Private data excluded
Y2	CSV exports: squads, fixtures, standings, results, supported stats, auction summary; neutralize formula injection (prefix = + - @ tab)	CSV	Y1	Injection test passes
Y3	PDF exports on demand (streamed), pagination and fonts	PDF	Y1	Opens correctly
Y4	Export UI and size limits/rate limits	UI	Y2	Large tournaments OK

Phase Z: Privacy controls, account lifecycle, suspensions and administrative workflows

Prereq: G, J, M. Completion: personal data handling matches policy.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
Z1	History-visibility setting honored by all public endpoints and exports	Visibility	G1, X	Tests per endpoint
Z2	Deletion processing: anonymize user shell, scrub email/credentials, keep IDs on results/audit; admin resolves request	Deletion	G12	Results intact; PII gone
Z3	Organizer suspension/removal of a player with reason and side effects on eligibility, teams, fixtures	Removal	K, M	History preserved
Z4	Admin user management, tournament suspension, exceptional interventions (each audited with reason)	Admin tools	G	Audit rows complete
Z5	Recovery workflow hardening and UI	Recovery UI	G11	Walkthrough

Phase AA: Audit-log viewing and administrative oversight

Prereq: all audited modules. Completion: logs searchable by role-appropriate scope.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
AA1	Admin audit viewer (filter by actor, action, entity, date; pagination)	Admin viewer	G13	Fast on 100k rows
AA2	Organizer private tournament log (scoped to own tournaments)	Organizer viewer	AA1	No cross-tournament leakage
AA3	Audit coverage review against Blueprint section 16	Coverage matrix	AA1	Every listed action logged

Phase AB: Accessibility, responsive design, performance and final polish

Completion: usable on phones, keyboard and screen readers; acceptable performance.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
AB1	Responsive pass for all pages at 360, 768, 1280 widths	Responsive	W	No horizontal scroll
AB2	Accessibility pass (labels, focus, contrast, ARIA for live auction region, reduced motion)	A11y	W	axe checks clean
AB3	Performance pass (indexes, pagination, bundle splitting, image sizes, Lighthouse)	Performance	W	Budgets met
AB4	Final UI polish, copy review, empty/error states	Polish	AB1	Review done

Phase AC: Testing and quality assurance (introduced throughout)

Testing is not a final phase: each row says when to start. Everything runs in GitHub Actions from Phase D onward.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
AC1	Unit tests (Vitest): pure logic such as hashing, state machines, standings, bracket and round-robin generators	Fast logic safety net	Phase D	Starts D13; grows each phase
AC2	Backend API tests (Supertest) incl. validation, error format, rate limits	Contract confidence	Phase D	Every endpoint has happy + failure case
AC3	Database integration tests on in-memory replica set (transactions, unique indexes)	Integrity proof	Phase G	Index violations asserted
AC4	Role and ownership permission matrix (visitor, player, unapproved organizer, approved organizer, other organizer, admin)	Security baseline	Phase G	Matrix extended with every new route
AC5	Authentication and session tests (cookies, expiry, revocation, CSRF, brute force)	Auth safety	Phase G	Green
AC6	Registration capacity and duplicate tests under parallel requests	Concurrency proof	Phase J	N parallel requests => exact cap
AC7	Auction concurrency, simultaneous bids, countdown, undo, purse, squad-limit tests	Core trust	Phase M	Run repeatedly (loop 100x) to catch flakiness
AC8	Fixture generation tests for knockout, round-robin, groups (sizes, byes, odd counts)	Correct schedules	Phases N to P	Property-based where useful
AC9	Result confirmation and correction tests (atomicity, downstream effects)	Result integrity	Phases R, T	Rollback verified
AC10	Standings and qualification recalculation tests (ties, corrections)	Table correctness	Phases P, T	Snapshot diffs asserted
AC11	CSV validation and malformed-file tests	Import safety	Phase S	Fuzz corpus committed
AC12	Socket.IO connection, auth, room and reconnection tests	Realtime safety	Phases M, V	Reconnect resync passes
AC13	Frontend component and interaction tests (React Testing Library)	UI behaviour	Phase E	Each feature gets tests
AC14	End-to-end tests (Playwright): register/login, organizer apply, create tournament, register, auction smoke, result confirm	Workflow proof	Phase H onward	Smoke suite on every PR; full suite nightly/before release
AC15	Responsive and accessibility checks (axe, keyboard, 360px)	Inclusive UX	Phase AB (spot checks earlier)	No critical violations
AC16	Security testing: OWASP checklist, dependency audit, authz bypass attempts, upload abuse, header review	Hardening	Each phase + final review	npm audit reviewed; manual pen checklist done
AC17	Backup and restoration test	Recoverability	Phase AD	Restore drill succeeds into scratch DB

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
AC18	Load/soak sanity for auctions and spectators	Capacity	Phase V	Documented limits

Phase AD: Production deployment

Do not assume any free tier is permanent: re-check pricing, sleep/cold-start behaviour, WebSocket support and database limits on the day you choose.

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
AD1	Re-check current free/low-cost offerings of shortlisted hosts (frontend, backend, Atlas) and record limits: cold starts, bandwidth, storage, connections, execution time, backup policy	Hosting decision ADR	A7, AB	ADR lists limits and expected cost
AD2	Decide URL topology so cookies work: same-site domain/subdomains, or reverse proxy, or serve built client from the API	Topology decision	AD1	Login works in Safari/mobile test plan
AD3	Create production accounts/projects (host, Atlas prod project, optional domain, monitoring)	Prod accounts	AD1	Access verified; 2FA on all
AD4	Provision production Atlas cluster, dedicated prod DB user, restrict network access to host egress IPs or required ranges	Prod DB	AD3, C13	Connects only from allowed origins
AD5	Create production secrets (session secret, DB URI, admin bootstrap password, CORS origins) in provider secret manager	Secrets	AD3	No secrets in repo/logs
AD6	Deploy backend: build with tsc, start script, PORT binding, trust proxy, health check path	Backend live	AD4, AD5	/api/health ok on public URL
AD7	Run one-time production init: index sync + create-admin script (never seed demo data)	Admin created	AD6	Only one admin exists
AD8	Deploy frontend: build with production API URL, SPA fallback rewrite to index.html for deep links	Frontend live	AD6	Refreshing /t/123/fixtures works
AD9	Configure CORS/allowed origins, cookie flags (Secure, SameSite), Socket.IO transports and sticky/single-instance assumptions	Cross-origin config	AD6, AD8	Browser console clean
AD10	Verify Socket.IO in production (WebSocket upgrade, idle timeouts, reconnect) with a mock auction	Realtime verified	AD9	Auction runs 30+ min without drop
AD11	Production smoke test of auth, uploads, reports, and every major workflow (use checklist in Part 4)	Smoke test passed	AD10	Checklist all ticked
AD12	Set up logs, error monitoring (e.g. Sentry free tier), uptime/health monitor, alerts	Observability	AD6	Test alert received
AD13	Create backup job (mongodump to private storage), retention, and perform a restore into a scratch database	Backups proven	AD4	Restore verified, runbook written
AD14	GitHub Actions deploy workflow: on merge to main run tests -> deploy; manual approval for production; rollback instructions	Repeatable deploys	AD6, AD8	Deploy from clean commit works

Step	Task	Expected outcome / deliverable	Depends on	Done when (completion criterion)
AD15	Final release checklist (Blueprint section 24) and security review	Release sign-off	AD11 to AD14	All items ticked; v1.0.0 tag